Report

Computational Workflows for Biomedical Data

Paula París Bayarri Dingling Yang

October 25, 2026

1 Introduction

1.1 Pipelines

Biomedical data analysis requires consecutive steps of processing. Combine these steps to a standard pipeline not only improves efficiency and spares researchers from running each step manually, but also improves standardization and reproducibility. A common tool for managing data pipeline is Nextflow [1].

1.2 FAIR Principles and Reproducibility

FAIR is the acronym for Findable, Accessible, Interoperable, and Reusable; it is a set of guidelines for making research data and metadata reusable [2]. Findable means the metadata and data should be easy to find; Accessible asks that users should be able to access the data they find; Interoperable represents that the data should be interoperable with workflows for analysis. Reusable is the goal of FAIR principles, data should be reusable, and to achieve this, data and metadata should be well stored and well described.

Reproducibility means obtaining the same results when the same analysis is applied to the same input data [3].

1.3 nf-core

nf-core is a global community collaborating to build open-source Nextflow components and pipelines. It provides reusable modules, containerised software (e.g. with Docker), parameter schemas with input validation, and standardised reporting (e.g. MultiQC) [4].

1.4 RNA-seq

RNA sequencing quantifies gene expression by sequencing cDNA fragments. A RNA-seq pipeline takes the sequencing reads as input and expression estimation as output. A minimal RNA-seq pipeline inculdes five steps: quailty control, trimming, alignment, duplicate marking, and quantificaton. nf-core/rnaseq is a community pipeline that covers these core steps. [5].

2 Methods

2.1 Structure of an nf-core Pipeline

This project was developed using an nf-core template based on Nextflow. The pipeline is divided into different modules, and the main workflow connects them by passing the required data between the different steps through channels. The `main.nf` file initializes the pipeline, while the `workflows/` directory contains the main analysis workflow. The different tools used in the

pipeline are included as modules in the `modules/` directory. There are several configuration files stored in the `conf/` directory. These files are used to define parameters, resources and execution settings. Schema files are used to validate the input parameters and the structure of the samplesheet. In addition, some custom functions were added to our project for strandedness validation and TPM calculation.

2.2 Computational Environment and Software

This pipeline was developed from an nf-core template and executed using Nextflow 26.04.6. Test runs were executed locally using the `test,docker` profile. Docker containers provided the software environments required by individual processes, helping to keep dependencies and software versions consistent across runs. The test configuration limited each task to 4 CPUs, 4 GB of RAM and one hour of execution time. The main software tools used in the workflow are FastQC, fastp, HISAT2, SAMtools, RSeQC, featureCounts, Salmon and MultiQC. Python was also used for the alternative TPM calculation method.

Table 1: Software tools and versions used in the workflow

Tool	Version	Tool	Version
Nextflow	26.04.6	SAMtools	1.24
nf-core	4.1.0	RSeQC	5.0.4
Docker	29.8.0	featureCounts	2.1.1
FastQC	0.12.1	Salmon	2.7.0
fastp	1.3.6	Python	3.9.5
HISAT2	2.2.3	MultiQC	1.35

2.3 Pipeline Steps

The core workflow consists of five steps:

1. **Quality Control:** Quality control was performed to assess the quality of the sequencing reads. For this purpose, the nf-core FastQC module was used. FastQC evaluates several metrics related to sequence quality, sequence composition, and adapter content. It generates two output files for each analyzed FASTQ file: an HTML file for visualizing the quality control report, and a ZIP file containing the report data, summary files, and additional information. [6]

2. **Trimming:** Trimming is the process of removing poor-quality bases, typically at the ends of sequencing reads, as well as adapter sequences that may remain after library preparation[7]. This preprocessing step aims to improve the quality of the reads used in downstream analysis. For this purpose, the nf-core fastp module was used. This module provides quality statistics and an HTML report summarizing the trim results for each processed sample. [8]

3. **Alignment:** The sequencing reads are short fragments whose original genomic positions are unknown. Therefore, they need to be aligned to a reference genome in order to determine where each read most likely originated. For this purpose, the nf-core HISAT2 modules were used. Firstly, hisat2_extractsplicesites was used to extract known splice sites from the gene annotation [9]. Then, hisat2_build generated an index from the reference genome, allowing HISAT2 to efficiently search for the genomic positions corresponding to each read [10]. Finally, hisat2_align mapped the processed sequencing reads to the indexed reference genome while taking the splice-site information into account [11]. The alignment step generates a BAM (Binary Alignment Map) file containing the aligned reads and a

corresponding BAI index file, which allows efficient access to specific regions of the BAM file.

4. **Duplicate marking:** During library preparation, PCR amplification can generate multiple copies of the same original DNA fragment. After the alignment, these copies may appear as duplicate reads and can potentially bias downstream analyses if they are treated as independent observation. Then, HISAT2 produced BAM files containing the genomic positions and orientation of the aligned reads. These BAM files were then processed using several SAMtools modules before duplicate marking. First, samtools_sort sorted the reads by name so that paired reads were placed together [12]. Then samtools_fixmate added and corrected mate-related information required for duplicate detection [13]. The BAM files were subsequently sorted by genomic coordinates using samtools_sort a second time [12]. Duplicate reads were then identified and marked with samtools_markdup [14]. Finally, samtools_index generated an index for the processed BAM file [15], and samtools_flagstat produced summary statistics, including the number of reads marked as duplicates [16].

5. **Quantification and TPM:** In the final step, we quantified the expression level of each gene by assigning the reads in the BAM file to the genes in the reference GTF file, and then counted and normalized the expression of each gene using featureCounts [17].

   However we cannot directly compare the raw counts because longer genes and deeply sequenced samples have more reads, so the counts were normalised to transcripts per million, using a local module: $\mathrm{TPM}_i = \frac{c_i/\ell_i}{\sum_j c_j/\ell_j} \cdot 10^6$, where c_i is the read count and ℓ_i the length (in kb) of gene i. This formula corrects both the length of genes and the depth (number of reads) of samples. Finally, the output values are merged to a table, with rows the name of the genes and columns the sample names.

2.4 Extentions

Strandedness validation In the samplesheet, the strandedness column is important for the gene-counting step performed by featureCounts, since it determines how reads are assigned to genes according to their orientation. Four possible values can be specified: auto, forward, reverse, or unstranded. However, featureCounts requires an explicit strandedness setting and cannot directly use auto. Samples that were labelled as auto, were treated as reverse in the original quantification workflow. Therefore, an additional validation step was implemented to infer the strandedness when necessary. Strandedness validation was performed using the rseqc_inferexperiment module. This module requires a gene annotation in BED12 format. For the test runs, the GTF (Gene Transfer Format) annotation was used and converted to BED12 format using eautils_gtf2bed. The resulting BED12 file was then used together with the aligned reads to infer the library strandedness [18, 19]. RSeQC compares the orientation of the aligned reads with the strand information of the annotated transcripts. The proportions supporting the forward and reverse models were normalized by excluding reads for which strandedness could not be determined. Therefore, the following formulas were applied to determine which model the orientation of that strand is supporting:

$$\text{Forward fraction} = \frac{\text{forward}}{\text{forward} + \text{reverse}}$$

$$\text{Reverse fraction} = \frac{\text{reverse}}{\text{forward} + \text{reverse}}$$

If one orientation represents at least 80% of the informative reads, the sample was classified as forward or reverse. Values close to an even distribution were interpreted as unstranded, while intermediate cases were considered undetermined.

Salmon branch for resource comparison. To test whether featureCounts is the most resource-efficient choice for quantification, we implemented an alternative branch, replacing featureCounts with Salmon[20].

In the Salmon branch, the GTF annotation is first filtered to chromosomes present in the genome [21], and transcript sequences are extracted with gffread [22]. A decoy-aware Salmon index is then built from the transcriptome and genome [23], and salmon_quant estimates transcript abundances directly as TPM [24]. Finally, a local module merges the results of all samples into one table.

3 Results

3.1 Pipeline Overview

The workflow covers the main RNA-seq processing steps, from sequencing reads to gene quantificacion and TPM matrix generation. It also produces a MultiQC report summarizing quality-control and processing metrics. The complete workflow is shown in Figure 1.

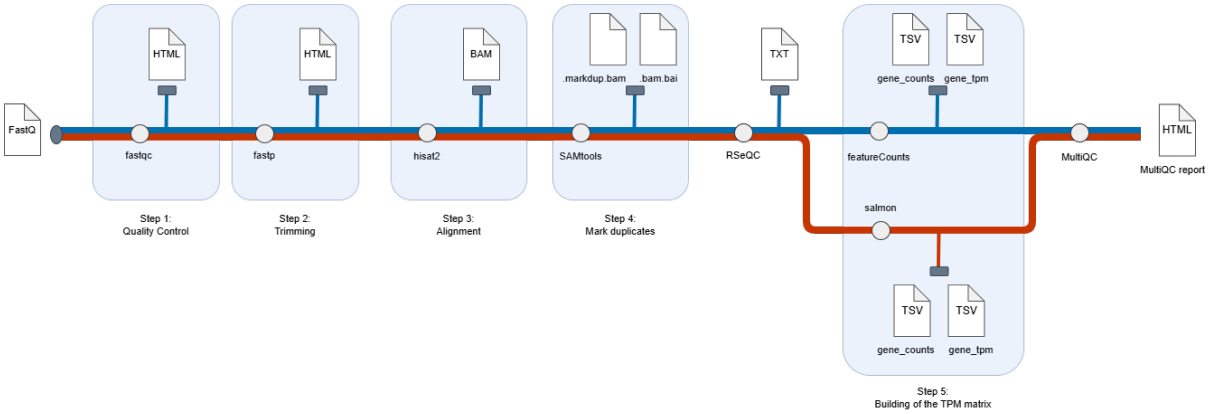


Figure 1: Overview of the developed workflow

Samplesheet We built a samplesheet using the test dataset provided by nf-core/rnaseq [25]. This dataset provides a reliable input for testing the workflow and also serves as an example of the expected input format for users.

3.2 Performance and Parallelization

Nextflow runs each sample independently, and only for the final step (step 5 described in Figure 1) - building TPM table, it waits for the results of all sample and combines it.

Table 2 summarizes the resource usage per step for the featureCounts version on the test dataset (five samples), recorded in the Nextflow execution trace.

Table 2: Resource usage per step of the featureCounts pipeline (test dataset, five samples), from the Nextflow execution trace. Task time is the `duration` field, which includes container start-up.

Step	Tasks	Summed task time	Peak memory
1. Quality control (FastQC)	5	38.7 s	340 MB
2. Trimming (fastp)	5	39.6 s	1.1 GB
3. Alignment (HISAT2: splice sites, index, align)	7	32.8 s	54 MB
4. Duplicate marking (SAMtools)	30	54.0 s	55 MB
Strandedness validation (gtf2bed, RSeQC)	6	18.5 s	45 MB
5. Quantification + TPM (featureCounts)	6	8.0 s	31 MB
MultiQC report	1	28.5 s	1.1 GB
Total	62	222 s	
Wall-clock time (actual duration)		89 s	

The summed task time is about 2.5 times the wall-clock time, showing the benefit of running samples in parallel. Duplicate marking was the most time-consuming step where as the trimming and multiQC had the highest memory peaks.

3.3 Strandedness validation

One of the five samples in the test samplesheet was labelled as "auto", meaning that an explicit strandedness value had to be determined before gene counting. The RSeQC validation showed a much higher reverse fraction than forward fraction, supporting a reverse-stranded library:

$$\text{Forward fraction} = \frac{0.0061}{0.0061 + 0.8499} \approx 0.007$$

$$\text{Reverse fraction} = \frac{0.8499}{0.0061 + 0.8499} \approx 0.993$$

Therefore, the automatically inferred value agreed with the assumption used for downstream quantification. For the remaining samples, which already had an explicit strandedness value, RSeQC was also used as a validation step. In all cases, the inferred orientation agreed with the declared strandedness, supporting the correctness of the samplesheet annotations. The strandedness results are summarized in Table 3.

Table 3: Comparison of declared, inferred and used strandedness values for the test samples

Sample	**Declared**	**Inferred**	**Used**
WAP1_IAA_30M_REP1	Reverse	Reverse	Reverse
RAP_UNINDUCED_REP1	Reverse	Reverse	Reverse
RAP1_UNIDUCED_REP2	Reverse	Reverse	Reverse
WT_REP1	Auto	Reverse	Reverse
WT_REP2	Reverse	Reverse	Reverse

3.4 Resource usage and TPM values comparison: featureCounts vs. Salmon

Resource usage. Table 4 compares the quantification step (step 5) of both versions. featureCounts finished in less than half the time of Salmon (8 s vs. 17 s, 53% faster) and needed a quarter of the memory (31 MB vs. 126 MB, 75% less). For this step, featureCounts is the more resource-efficient choice.

Table 4: Resource usage of the quantification step (step 5) only, taken from the Nextflow trace (five samples).

	Salmon	featureCounts	Saved
Runtime	17 s	8 s	53%
Peak memory	126 MB	31 MB	75%

Agreement of TPM values. To check that the cheaper method does not change the biological result, we compared the TPM values of both versions per sample (Figure 2, Table 5). The two methods agree closely: Pearson r ranged from 0.938 to 0.948 and Spearman ρ from 0.950 to 0.968.

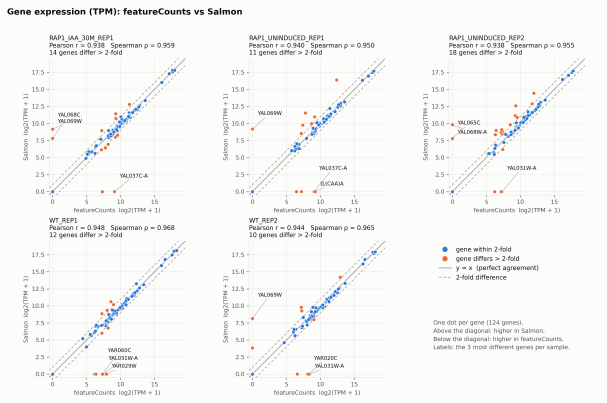


Figure 2: Comparison of gene expression (TPM) estimated by featureCounts and Salmon for each of the five samples. Each dot is one gene, plotted as $\log_2(\text{TPM} + 1)$ from featureCounts (x-axis) against Salmon (y-axis). The solid diagonal marks perfect agreement ($y = x$) and the dashed lines mark a two-fold difference.

Table 5: Correlation of TPM values between featureCounts and Salmon.

Sample	Pearson r	Spearman ρ
RAP1_IAA_30M_REP1	0.938	0.959
RAP1_UNINDUCED_REP1	0.940	0.950
RAP1_UNINDUCED_REP2	0.938	0.955
WT_REP1	0.948	0.968
WT_REP2	0.944	0.965

In retrospect Salmon is not the fairest choice for this validation, as it does not require the results from aligning and duplicates marking, instead processes data from trimming. But in this report, we still document what we have done, and hope it can at least justify that featureCount is a more efficient tool under the 5step schema than Salmon.

4 Discussion

4.1 Reproducibility and FAIR Pipelines in nf-core

As previously mentioned, nf-core is designed to support reproducible bioinformatics workflows. Since our pipeline was developed from an nf-core template, it benefits from many of the reproducibility features provided by the nf-core structure.

Fair principles aim to make scientific workflows findable, accessible, interoperable and reusable. In our project, this is supported by publishing the pipeline on GitHub, using standard bioinformatics file format, documentaing the required inputs and parameters, and providing a test samplesheet that can be used as an example for future users.

4.2 Limitations and Outlook

However, our workflow would still benefit from broader testing, more complete documentation and further validation with additional datasets before it can be considered fully robust and reusable. Our current limitation is that it has mainly been tested using the predefined test samplesheet, although smaller variants of the test samplesheet were also evaluated successfully. It has not yet been validated with a different and larger dataset, such as the dataset used in the previous week's assignment. Testing the pipeline with additional datasets of different sizes and characteristics would therefore be an important next step to evaluate the robustness and general applicability. On the other hand, the workflow was successfully executed on different computers, which provides evidence that the computational environment is reproducible across systems.

A specific limitation is on consumption validation. As mentioned in the section "2.4 Extension", Salmon is not a bulletproof comparison to featureCounts. If we were to refine the experiment, we would first try other alternatives of featureCounts which also runs on top of the result of aligned BAM files, such as HTSeq-count or StringTie. Moreover, accuracy is important. In our extension of quantification tool validation, we only observed that the results of two variants didn't diverge too far, however agreements between tools doesn't imply accuracy, because both tools could share the same bias. Therefore, evaluating the results of quantification is also a topic worth further research, which would allow a better choice.

References

[1] Seqera. *Nextflow Documentation: Overview*. `https://docs.seqera.io/nextflow/overview`. Accessed: 2026-10-20.

[2] National Library of Medicine. *Definition of FAIR Data. Common Data Elements: Standardizing Data Collection*. URL: `https://www.nlm.nih.gov/oet/ed/cde/tutorial/02-200.html` (visited on 10/20/2026).

[3] Nature Communications. "Reproducibility and transparency: what's going on and how can we help". In: *Nature Communications* 16 (2025), p. 1082. DOI: `10.1038/s41467-024-54614-2`.

[4] nf-core community. *What is nf-core?* URL: `https://nf-co.re/docs/get_started/nf-core` (visited on 10/20/2026).

[5] nf-core community. *nf-core/rnaseq: Introduction (version 3.14.0)*. DOI: `10.5281/zenodo.1400710`. URL: `https://nf-co.re/rnaseq/3.14.0/` (visited on 10/20/2026).

[6] nf-core. *FastQC module*. URL: `https://nf-co.re/modules/fastqc/` (visited on 10/18/2026).

[7] Jay Z. Parrish Claire R. Williams Alyssa Baccarella and Charles C. Kim. "Trimming of sequence reads alters RNA-Seq gene expression estimates". In: *BMC Bioinformatics* 17 (2016). DOI: `10.1186/s12859-016-0956-2`.

[8] nf-core. *fastp module*. URL: `https://nf-co.re/modules/fastp/` (visited on 10/18/2026).

[9] nf-core. *HISAT2 extractsplicesites module*. URL: `https://nf-co.re/modules/hisat2_extractsplicesites/` (visited on 10/18/2026).

[10] nf-core. *HISAT2 build module*. URL: `https://nf-co.re/modules/hisat2_build/` (visited on 10/18/2026).

[11] nf-core. *HISAT2 align module*. URL: https://nf-co.re/modules/hisat2_align/ (visited on 10/18/2026).

[12] nf-core. *SAMtools sort module*. URL: https://nf-co.re/modules/samtools_sort/ (visited on 10/18/2026).

[13] nf-core. *SAMtools fixmate module*. URL: https://github.com/nf-core/modules/tree/master/modules/nf-core/samtools/fixmate (visited on 10/18/2026).

[14] nf-core. *SAMtools markdup module*. URL: https://nf-co.re/modules/samtools_markdup/ (visited on 10/18/2026).

[15] nf-core. *SAMtools index module*. URL: https://nf-co.re/modules/samtools_index/ (visited on 10/18/2026).

[16] nf-core. *SAMtool flagstat module*. URL: https://nf-co.re/modules/samtools_flagstat/ (visited on 10/18/2026).

[17] nf-core. *featureCounts module*. URL: https://nf-co.re/modules/subread_featurecounts (visited on 10/18/2026).

[18] nf-core. *RSeQC inferexperiment module*. URL: https://nf-co.re/subworkflows/bam_rseqc/ (visited on 10/18/2026).

[19] nf-core. *ea-utils gtf2bed module*. URL: https://github.com/nf-core/modules/tree/master/modules/nf-core/ea-utils/gtf2bed (visited on 10/18/2026).

[20] Rob Patro et al. "Salmon provides fast and bias-aware quantification of transcript expression". In: *Nature Methods* 14.4 (2017), pp. 417–419. DOI: 10.1038/nmeth.4197.

[21] nf-core community. *nf-core module: custom/gtffilter*. URL: https://nf-co.re/modules/custom_gtffilter/ (visited on 10/20/2026).

[22] nf-core community. *nf-core module: gffread*. URL: https://nf-co.re/modules/gffread/ (visited on 10/20/2026).

[23] nf-core community. *nf-core module: salmon$_i$ndex*. URL: https://nf-co.re/modules/salmon_index (visited on 10/20/2026).

[24] nf-core community. *nf-core module: salmon$_q$uant*. URL: https://nf-co.re/modules/salmon_quant (visited on 10/20/2026).

[25] nf-core. *RNAseq pipeline*. URL: https://nf-co.re/rnaseq/3.27.0/ (visited on 10/18/2026).